

0484

Bases de datos

Clase 5

05

Manipulación y transacciones



El lenguaje SQL

- Para **definir la estructura** de la BD:
DDL (Data Definition Language)
 - Definición de tablas y otros objetos de la BD
 - Ejemplo: CREATE TABLE....
- Para **consultar y manipular** la BD:
DML (Data Manipulation Language)
 - Consultas de datos
 - Actualización, inserción y eliminación de datos
- Para **controlar el acceso a los datos** DCL (Data Control Language)
 - Controlar la seguridad de los datos y cuentas de usuario

Cláusula INSERT. Formato 1

```
INSERT INTO NombreTabla [(Campo1, ..., CampoN)] VALUES (Valor1, ..., ValorN)
```

- Los valores (Valor1,...,ValorN) se deben corresponder con cada uno de los campos que aparecen en la lista de campos (Campo1,...,CampoN) tanto en:
 - el **tipo de dato** que contienen
 - como en el **orden** en el que se van a asignar.
- Si un campo no está en la lista, se almacenará dentro de éste el valor NULL.
- Si un campo está definido como **NOT NULL** (es decir, que no admite nulos o valores vacíos), **debemos especificarlo siempre en la lista de campos a insertar**. De no hacerlo así se producirá un error al ejecutar la correspondiente instrucción INSERT

Cláusula INSERT. Formato 1

Ejemplo inserción en la tabla trabajadores de la base de datos empresa

```
INSERT INTO trabajadores VALUES ("33112222A","Nualart Vives, Carlos", "Barcelona",10, 30000, 4);
```

```
INSERT INTO trabajadores VALUES ("33112222A","Nualart Vives, Carlos", "Barcelona",10, 30000, 4);
```

La tabla tiene la siguiente estructura:

```
CREATE TABLE IF NOT EXISTS trabajadores (  
  dni VARCHAR(9) PRIMARY KEY ,  
  nombre VARCHAR(50),  
  ciudad VARCHAR(40),  
  antigüedad INT(2),  
  salario decimal (8,2),  
  departamento int not null,  
  foreign key (departamento) REFERENCES departamento(numdep));
```

Cláusula INSERT. Formato 1

Ejemplo de inserción en una tabla con un campo autonumérico:

```
INSERT INTO departamento VALUES (0, 'Marketing');  
select * from departamento;
```

```
CREATE TABLE IF NOT EXISTS departamento (  
  numdep int not null auto_increment,  
  nombredep varchar(20) not null unique,  
  primary key (numdep));
```

Para insertar valores autonuméricos ponemos 0. El sistema calculará automáticamente el valor numérico correspondiente

No es necesario especificar este tipo de campos en las instrucciones INSERT. Por ello, por ejemplo, para insertar un nuevo departamento en una tabla “departamento”, que tiene un campo clave autonumérico y otro de tipo varchar con la descripción, lo único que hay que hacer es indicar el nombre del campo al que sí se le asigna valor:

```
INSERT INTO departamento (nombredep) VALUES ('Ventas');  
select * from departamento;
```

Ahorrándonos tener que escribir el nombre del campo

Cláusula INSERT. Formato 2

Podemos especificar el nombre y valor de la columnas explícitamente con SET

```
INSERT INTO trabajadores SET dni="22334455C", nombre="Perez Cano, Isabel", ciudad="Madrid",  
antigüedad=2, salario=36000, departamento=3;
```

	dni	nombre	ciudad	antigüedad	salario	departamento
	11112266C	Llamas Rocasolano, Isabel	Barcelona	13	28000.00	3
	22112222A	Roca Benítez, Elena	Sevilla	6	35000.00	4
	22334455C	Perez Cano, Isabel	Madrid	2	36000.00	3
	33112222C	Muñoz Casas, Montse	NULL	7	40000.00	5

Cláusula INSERT. Formato 3

Podemos insertar en una tabla múltiples registros procedentes de otra tabla, obteniéndolos a partir de una consulta SELECT.

La sintaxis genérica de la instrucción que nos permite insertar registros procedentes de una consulta SELECT es la siguiente:

```
INSERT INTO NombreTabla [(Campo1, ..., CampoN)] SELECT ...
```

El SELECT se indica a continuación de la lista de campos, y en lugar de especificar los valores con VALUES, se indica una consulta de selección.

Es indispensable que los campos devueltos por esta instrucción SELECT **coincidan en número y tipo de datos con los campos que se han indicado antes**, o se producirá un error de ejecución y no se insertará ningún registro.

Ejemplo:

```
-- Ejemplo 4 insercción con instrucción select

CREATE TABLE trabajadoresbarcelona LIKE trabajadores;
DESC trabajadoresbarcelona;

INSERT into trabajadoresbarcelona
SELECT * from trabajadores WHERE ciudad="barcelona";

select * from trabajadoresbarcelona;
```

dni	nombre	ciudad	antiquedad	salario	departamento
11112222A	Rojo Iglesias, Marta	Barcelona	12	60000.00	2
11112266C	Llamas Rocasolano, Isabel	Barcelona	13	28000.00	3

El SELECT se indica a continuación de la lista de campos, si se pone, y en lugar de especificar los valores con VALUES, se indica una consulta de selección.

Es indispensable que los campos devueltos por esta instrucción SELECT **coincidan en número y tipo de datos con los campos que se han indicado antes**, o se producirá un error de ejecución y no se insertará ningún registro.

La información introducida en las tablas se puede modificar mediante la cláusula UPDATE.

Su sintaxis es:

UPDATE NombreTabla

SET Campo1=Valor1,...,CampoN=ValorN WHERE

Condición

Ejemplo (Departamentos):

```
UPDATE departamento  
SET nombredep = 'R.Humanos'  
WHERE numdep = '2';
```

	numdep	nombredep
▶	4	Comercial
	1	Contabilidad
	5	Facturación
	3	Informática
	2	Recursos Humanos

	numdep	nombredep
▶	4	Comercial
	1	Contabilidad
	5	Facturación
	3	Informática
	2	R.Humanos

Cláusula UPDATE

Subimos el salario de trabajadores del departamento 2 un 10%.

```
-- Ejemplo1 update
-- Subimos un 10% el salario de los trabajadores del departamento 2

SELECT * from trabajadores where departamento=2;
UPDATE trabajadores SET salario=salario+(salario*0.10) where departamento=2;
```

	dni	nombre	ciudad	antigüedad	salario	departamento
▶	11112233A	Perez Carrillo, Iván	Bilbao	5	48000.00	2
	11112244B	Torres Marqués, Fernando	Madrid	11	55000.00	2

	dni	nombre	ciudad	antigüedad	salario	departamento
▶	11112233A	Perez Carrillo, Iván	Bilbao	5	52800.00	2
	11112244B	Torres Marqués, Fernando	Madrid	11	60500.00	2
*	NULL	NULL	NULL	NULL	NULL	NULL

Cláusula UPDATE

```
-- Ejemplo2 update
-- Añadimos un campo llamado dietas a la tabla trabajadores
-- Ponemos un 2% de su salario
-- A los trabajadores de madrid con una antigüedad superior a 6 años.
```

```
select * from trabajadores
where ciudad="Madrid" and antiguedad>6;
```

```
alter table trabajadores
add dietas decimal (8,2);
```

```
desc trabajadores;
```

```
UPDATE trabajadores SET dietas=salario*0.02
where ciudad="Madrid" and antiguedad>6;
```

[illegible]

Instrucción DELETE

La instrucción DELETE permite eliminar uno o múltiples registros de una tabla Su sintaxis es:

DELETE [FROM] NombreTabla WHERE Condición

ATENCIÓN: Es MUY IMPORTANTE poner siempre una cláusula WHERE. De modo contrario se borrarían todos los datos de la tabla!!

```
/* Ejemplo 1 */  
-- Borramos el trabajador con dni '22334455C'  
  
SELECT * from trabajadores;  
DELETE FROM trabajadores where dni='22334455C';
```

Borrado y modificaciones e Integridad referencial

La integridad referencial es un sistema de reglas que utilizan la mayoría de las bases de datos relacionales para asegurarse que los registros de tablas relacionadas son válidos y que no se borren o cambien datos relacionados de forma accidental produciendo errores de integridad.

Cláusula ON DELETE...ON CASCADE

Cuando existe **integridad referencial** entre dos tablas, en la definición de una clave ajena se suele indicar que sucede cuando el valor del campo referenciado se modifica.

Ejemplo:

```
ALTER TABLE trabajadores ADD CONSTRAINT fk_departamento  
FOREIGN KEY departamento REFERENCES departamento(numdep)  
ON DELETE NO ACTION ON UPDATE CASCADE;
```

Posibilidades en la definición de la integridad referencial:

- **SET NULL:** Permite borrar o actualizar un registro en la tabla “padre” poniendo a NULL el campo o campos relacionados en la tabla “hija”
- **CASCADE :** Propaga las modificaciones o borrados a los registros de las tablas hijas relacionados con la tabla padre
- **NO ACTION/RESTRICT:** Se impide la eliminación o modificación de un registro en la tabla “padre” si tiene registros relacionados en la tabla “hija”.

Cláusula ON DELETE...ON CASCADE

```
/*Integridad Referencial*/
-- Borramos la clave foranea.alter
ALTER TABLE trabajadores
DROP FOREIGN KEY fk_departamento;

-- Añadimos la nueva clave foránea con las restricciones.
ALTER TABLE trabajadores
ADD CONSTRAINT fk_departamento FOREIGN KEY (departamento)
REFERENCES departamento(numdep)
ON DELETE NO ACTION
ON UPDATE CASCADE;

-- Probamos el DELETE (da error)
-- Probamos el UPDATE (si funciona)

DELETE from departamento where nombredep='Recursos Humanos';
UPDATE departamento SET numdep='20' WHERE nombredep='Recursos Humanos';
SELECT * from trabajadores;
```

Error Code: 1451. Cannot delete or update a parent row: a foreign key constraint fails ('empresa`.`trabajadores`, CONSTRAINT `fk_departamento`')

	dni	nombre	ciudad	antigüedad	salario	departamento
▶	11112222A	Rojo Iglesias, Marta	Barcelona	12	60000.00	20
	11112222C	Gomez Corachán, Manuel	Madrid	15	22000.00	1
	11112233A	Perez Carrillo, Iván	Bilbao	5	48000.00	20
	11112244B	Torres Marqués, Fernando	Madrid	11	55000.00	20

Transacciones.

Una transacción es un conjunto de órdenes (en nuestro caso comandos SQL) que se ejecutan de manera atómica o indivisible, es decir se ejecutan todas o ninguna.

Deben cumplir las cuatro propiedades ACID:

- **Atomicidad** : Asegura que se realizan todas las operaciones o ninguna, no puede quedar a medias
- **Consistencia o integridad** : Asegura que solo se empieza lo que se puede acabar.
- **Aislamiento**: Asegura que ninguna operación afecta a otras pudiendo causar errores
- **Durabilidad**: Asegura que una vez realizada la operación, ésta no podrá cambiar y permanecerán los cambios

Secuencia de transacción en MYSQL.

1. Iniciar una transacción con el uso de la sentencia `START TRANSACTION` o `BEGIN`
2. Actualizar, insertar o eliminar registros en la base de datos
3. Si se quieren los cambios a la base de datos, completar la transacción con el uso de la sentencia `COMMIT`. Únicamente cuando se procesa un `COMMIT` los cambios hechos por las consultas serán permanentes
4. Si sucede algún problema, podemos hacer uso de la sentencia `ROLLBACK` para cancelar los cambios que han sido realizados por las consultas que han sido ejecutadas hasta el momento

1. Comprobamos el estado de la variable autocommit.

```
SHOW VARIABLES LIKE 'autocommit';
```

	Variable name	Value
▶	autocommit	ON

- **Con valor=1.** Está activada lo que implica que cada comando SQL es en si mismo una transacción. El comando START TRANSACTION desactiva este comportamiento.
- **Con valor=0** . Se desactiva lo que hace que sólo los comandos COMMIT y ROLLBACK confirmen o revoquen una sentencia SQL

2. Creamos una tabla, en la base de datos test, del tipo InnoDB e insertamos algunos datos

Para crear una tabla InnoDB, procedemos con el código SQL estándar CREATE TABLE, pero debemos especificar que se trata de una tabla del tipo InnoDB (TYPE= InnoDB).

```
DROP DATABASE IF EXISTS test ;  
CREATE DATABASE test;  
USE test;
```

```
/* Importante! El tipo de datos debe ser InnoDB */  
CREATE TABLE ciudades  
(nombre VARCHAR(30) PRIMARY KEY) Engine='InnoDB';
```

```
/* Insertamos 3 valores, uno en cada fila */  
INSERT INTO ciudades VALUES ('Barcelona'),('Madrid'),('Sevilla');  
SELECT * FROM ciudades;
```

	nombre
▶	Barcelona
	Madrid
	Sevilla
*	HULL

Transacciones - ROLLBACK.

3. Una vez cargada la tabla iniciamos una transacción

```
BEGIN;  
INSERT INTO ciudades VALUES('Bilbao');
```

Si en este momento ejecutamos un ROLLBACK, la transacción no será completada y los cambios realizados sobre la tabla no tendrán efecto.

```
use test;  
BEGIN;  
  
INSERT INTO ciudades VALUES ('Bilbao');  
SELECT * FROM ciudades;  
/* La transacción no está completada y la vamos a deshacer usando  
ROLLBACK*/  
ROLLBACK;  
  
/* Comprobamos que el último valor no está en la tabla*/  
SELECT * FROM ciudades;
```

nombre
Barcelona
Bilbao
Madrid
Sevilla
NULL

nombre
Barcelona
Madrid
Sevilla
NULL

```
/* ¿Qué ocurre si antes de finalizar la transacción perdemos
la conexión con el servidor? */
use test;
BEGIN;

INSERT INTO ciudades VALUES ('Bilbao');
SELECT * FROM ciudades;
/* La transacción no está completada y simulamos que perdemos la
conexión con el servidor AQUÍ*/

/* Comprobamos que el último valor no está en la tabla*/
use test;
SELECT * FROM ciudades;
```

	nombre
▶	Barcelona
	Madrid
	Sevilla
*	NULL

Cuando se procesa un COMMIT los cambios hechos por las consultas serán permanentes.

```
/* Ejemplo COMMIT. Repetimos de nuevo la sentencia INSERT
pero hacemos COMMIT antes de perder la conexión con el servidor */
USE test;
BEGIN;

INSERT INTO ciudades VALUES ('Bilbao');
COMMIT;
/* Aquí simular una desconexión del servidor */

/* Podemos comprobar que el último valor ahora sí que está en la tabla */
use test;
SELECT * FROM ciudades;
```

	nombre
▶	Barcelona
	Bilbao
	Madrid
	Sevilla
*	NULL

Otros comandos de transacciones.

- **SAVEPOINT id**

Es una instrucción que permite nombrar cierto paso en un conjunto de instrucciones de una transacción

- **ROLLBACK TO SAVEPOINT id**

Permite deshacer el conjunto de operaciones realizadas a partir del identificador de savepoint

- **RELEASE SAVEPOINT id**

Elimina o libera el savepoint creado

Cualquier operación COMMIT o ROLLBACK sin argumentos eliminará todos los savepoints creados

Transacciones - SAVEPOINT

```
/* Ejemplo SAVEPOINT. Permite nombrar un cierto paso de la transacción */  
USE test;  
DELETE FROM ciudades;  
BEGIN;  
INSERT INTO ciudades VALUES ('Barcelona');  
SAVEPOINT PAS01; -- punto de control de nombre PAS01  
INSERT INTO ciudades VALUES ('Madrid'),('Sevilla'),('Bilbao');  
SELECT * FROM ciudades;  
  
ROLLBACK TO PAS01; -- deshacemos las operaciones hasta el PAS01  
SELECT * FROM ciudades;  
COMMIT;
```

	nombre
▶	Barcelona
	Bilbao
	Madrid
	Sevilla
*	NULL

	nombre
▶	Barcelona
*	NULL

Transacciones

Bloquear Tablas. WRITE

Bloqueo WRITE. El cliente o la sesión que tiene el bloqueo puede leer y escribir en la tabla, pero ningún otro cliente podrá acceder a ella o bloquearla.

```
/* Ejemplo Bloqueos de tablas. WRITE
-En el caso de escritura ningún otro cliente podrá acceder hasta que desbloqueemos */

USE test;
DELETE FROM ciudades;
INSERT INTO ciudades VALUES ('Madrid'),('Sevilla'),('Bilbao'),('Barcelona');
select * from ciudades;

USE test;
LOCK TABLES ciudades WRITE;
UPDATE ciudades SET nombre="BCN" WHERE nombre="Barcelona";

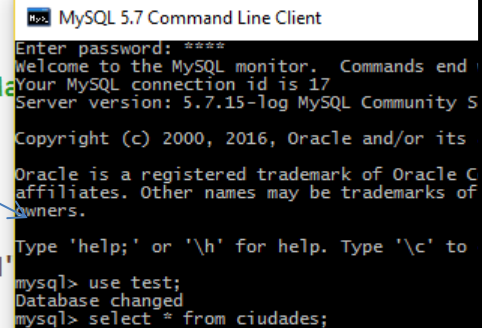
UNLOCK TABLES;
```

```
USE test;

INSERT INTO ciudades VALUES ('Ma
select * from ciudades;

USE test;
LOCK TABLES ciudades WRITE;
UPDATE ciudades SET nombre="BCN"

UNLOCK TABLES;
```



```
MySQL 5.7 Command Line Client
Enter password: ****
Welcome to the MySQL monitor.  Commands end with ; or \g
Your MySQL connection id is 17
Server version: 5.7.15-log MySQL Community Server - GPL
Copyright (c) 2000, 2016, Oracle and/or its affiliates. Other names may be trademarks of their owners.
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> use test;
Database changed
mysql> select * from ciudades;
```


Transacciones – Bloquear Tablas READ

Bloqueo READ. El cliente o sesión que tiene el bloqueo y los otros clientes puede LEER la tabla. Pero ninguno puede modificar el contenido, es decir, escribir en la tabla.

```
/* Ejemplo Bloqueos de tablas. READ
-En el caso de lectura los clientes pueden acceder pero no modificar */

USE test;
LOCK TABLES ciudades READ;

UNLOCK TABLES;
```

```
USE test;
LOCK TABLES ciudades READ;

UNLOCK TABLES;
```

```
5 rows in set (12 min 16.58 sec)
mysql>
mysql> select * from ciudades;
+-----+
| nombre |
+-----+
| BCN    |
| Bilbao|
| Madrid|
| Sevilla|
| Valencia|
+-----+
5 rows in set (0.00 sec)
mysql>
```

06

Programación de bases de datos



La programación en MySQL permite añadir lógica directamente dentro del servidor de bases de datos.

Esto significa que no todas las operaciones deben realizarse desde el lenguaje de programación externo (Java, Python, PHP...), sino que el propio SGBD puede ejecutar: Automatizaciones, Validaciones, Cálculos complejos, Auditorías, Gestión de errores

Esto mejora la velocidad porque la lógica se ejecuta **sin enviar tantos datos al cliente**, y aumenta la coherencia porque todas las aplicaciones comparten la misma lógica de negocio.

Tipos más importantes:

- **Procedimientos almacenados** → bloques reutilizables que ejecutan operaciones.
- **Funciones** → devuelven un valor calculado.
- **Triggers (disparadores)** → se ejecutan automáticamente al modificar datos.
- **Cursores** → permiten recorrer resultados fila a fila.
- **HANDLER** → control de errores.

Sintaxis básica de bloques de código

Todo procedimiento o función sigue una estructura definida por el bloque BEGIN ... END.
El cambio de delimitador (DELIMITER //) es necesario para evitar que el ; finalice la definición del procedimiento prematuramente.
A continuación, se muestra la sintaxis genérica:

```
DELIMITER //  
CREATE PROCEDURE nombre_procedimiento()  
BEGIN  
    -- Instrucciones SQL  
END;  
//  
DELIMITER ;
```

Las variables permiten almacenar valores temporales para usar dentro de bloques, procedimientos o consultas más complejas. Tipos:

- **Variables del sistema**

Configuran el comportamiento global o por sesión del servidor (autocommit, timezone...).

- **Variables de usuario (@variable)**

Se crean sobre la marcha y duran mientras la sesión esté activa.

- **Variables locales**

Solo existen dentro de bloques BEGIN...END y necesitan DECLARE. Se usan para cálculos intermedios, control de bucles, conteos y resultados lógicos.

```
-- Ejemplo Variables de usuario.
```

```
SET @salario_medio = (SELECT AVG(salario) FROM trabajadores);  
SELECT @salario_medio;
```

Variables locales en bloques

Las variables locales permiten guardar información temporal dentro de procedimientos, funciones o triggers. Son fundamentales cuando hay que procesar datos paso a paso.

- Se declaran con DECLARE
- Deben ir al **inicio** del bloque
- Solo existen mientras dure el BEGIN...END

```
-- Ejemplo 1 Variables locales
DELIMITER //
CREATE PROCEDURE mostrar_max_salario()
BEGIN
    DECLARE max_sal DECIMAL(8,2);
    SELECT MAX(salario) INTO max_sal FROM trabajadores;
    SELECT max_sal AS salario_maximo;
END;
//
DELIMITER ;
CALL mostrar_max_salario();
```

Procedimientos almacenados

Un procedimiento ejecuta instrucciones complejas y puede modificar datos. Son ideales para automatizar procesos. Les podemos pasar parámetros y se ejecutan haciendo un call.

```
-- Procedimiento almacenado. Subir salario de un departamento.
DELIMITER //
CREATE PROCEDURE subir_salario_departamento(
    IN p_dep INT,
    IN p_porcentaje DECIMAL(5,2)
)
BEGIN
    UPDATE trabajadores
    SET salario = salario * (1 + p_porcentaje/100)
    WHERE numdep = p_dep;
END;
//
DELIMITER ;
CALL subir_salario_departamento(2,5);
select * from trabajadores;
```


Condicionales IF y CASE

Permite ejecutar bloques de código según se cumpla una condición.

```
IF condicion THEN
    -- Instrucciones
ELSEIF otra_condicion THEN
    -- Instrucciones
ELSE
    -- Instrucciones
END IF;
```

```
CASE valor
    WHEN 1 THEN SET mensaje = 'Uno';
    WHEN 2 THEN SET mensaje = 'Dos';
    ELSE SET mensaje = 'Otro';
END CASE;
```

```
CASE
    WHEN edad < 18 THEN SET categoria = 'Menor';
    WHEN edad < 65 THEN SET categoria = 'Adulto';
    ELSE SET categoria = 'Senior';
END CASE;
```


IF permite ejecutar código dependiendo de una condición.
Solo puede usarse en procedimientos, funciones o triggers.

```
-- Estructura IF
DELIMITER //
CREATE PROCEDURE clasificar_salario(IN p_dni VARCHAR(9))
BEGIN
    DECLARE s DECIMAL(8,2);

    SELECT salario INTO s
    FROM trabajadores
    WHERE dni = p_dni;

    IF s >= 50000 THEN SELECT 'Salario alto' AS categoria;
    ELSEIF s >= 30000 THEN SELECT 'Salario medio' AS categoria;
    ELSE SELECT 'Salario bajo' AS categoria;
    END IF;
END;
//
DELIMITER ;
CALL clasificar_salario('11112222A');
```

Instrucción CASE

CASE sirve para categorizar datos dentro de un SELECT sin usar procedimientos.

```
-- Instrucción CASE
SELECT nombre, antigüedad,
CASE
    WHEN antigüedad >= 10 THEN 'Muy veterano'
    WHEN antigüedad >= 5 THEN 'Veterano'
    ELSE 'Nuevo'
END AS categoria
FROM trabajadores;
```

	nombre	antigüedad	categoría
►	Rojo Iglesias, Marta	12	Muy veterano
	Gomez Corachán, Manuel	15	Muy veterano
	Perez Carrillo, Iván	5	Veterano
	Torres Marqués, Fernando	11	Muy veterano
	Rubio Sánchez, María	4	Nuevo
	Llamas Rocasolano, Isabel	13	Muy veterano
	Roca Benítez, Elena	6	Veterano
	Perez Cano, Isabel	2	Nuevo
	Nualart Vives, Carlos	10	Muy veterano
	Muñoz Casas, Montse	7	Veterano

Iterativas: Loop, while, repeat

Los bucles permiten repetir código. Solo se usan dentro de procedimientos.

```
[etiqueta:] LOOP
  -- Instrucciones
  LEAVE etiqueta;
END LOOP;
```

Ejemplo:

```
sql

etiqueta: LOOP
  SET contador = contador + 1;
  IF contador >= 10 THEN
    LEAVE etiqueta;
  END IF;
END LOOP;
```

```
WHILE condicion DO
  -- Instrucciones
END WHILE;
```

Ejemplo:

```
sql

WHILE i < 5 DO
  SET suma = suma + i;
  SET i = i + 1;
END WHILE;
```

```
REPEAT
  -- Instrucciones
UNTIL condicion
END REPEAT;
```

Ejemplo:

```
sql

REPEAT
  SET total = total + cantidad;
  SET i = i + 1;
UNTIL i > 10
END REPEAT;
```

Bucles WHILE

Los bucles permiten repetir código. Solo se usan dentro de procedimientos.

```
-- Bucles While
DELIMITER //
CREATE PROCEDURE contar_hasta(IN limite INT)
BEGIN
    DECLARE c INT DEFAULT 1;

    WHILE c <= limite DO
        SELECT c;
        SET c = c + 1;
    END WHILE;
END;
//
DELIMITER ;
call contar_hasta(10);
```

	c
▶	10

Procedimientos y funciones

Los procedimientos almacenados y las funciones definidas por el usuario (UDF) son bloques de código que se guardan en el servidor MySQL y permiten centralizar la lógica de negocio en la base de datos.

Característica	Procedimiento (PROCEDURE)	Función (FUNCTION)
Tipo de retorno	No devuelve valor directamente	Devuelve un valor único obligatorio
Uso en consultas SQL	No se puede usar directamente	Se puede usar dentro de SELECT, WHERE...
Parámetros admitidos	IN, OUT, INOUT	Solo IN
Invocación	CALL nombre()	SELECT nombre()
Aplicación	Automatización de tareas complejas	Cálculo o transformación de un dato

Procedimientos almacenados

Un procedimiento ejecuta instrucciones complejas y puede modificar datos. Son ideales para automatizar procesos. Les podemos pasar parámetros y se ejecutan haciendo un call.

```
-- Procedimiento almacenado. Subir salario de un departamento.
DELIMITER //
CREATE PROCEDURE subir_salario_departamento(
    IN p_dep INT,
    IN p_porcentaje DECIMAL(5,2)
)
BEGIN
    UPDATE trabajadores
    SET salario = salario * (1 + p_porcentaje/100)
    WHERE numdep = p_dep;
END;
//
DELIMITER ;
CALL subir_salario_departamento(2,5);
select * from trabajadores;
```

Funciones definidas por el usuario

- **Devuelve un único valor** como resultado.
- **Admite parámetros de entrada**, pero **no** tiene parámetros de salida (a diferencia de los procedimientos).
- Se puede utilizar en cualquier lugar donde MySQL permita una expresión (por ejemplo, en un SELECT o en una cláusula WHERE).
- Siempre debe contener una cláusula RETURN.

Parámetros

- RETURNS: tipo de dato que devuelve la función.
- DETERMINISTIC / NOT DETERMINISTIC: indica si la función siempre devuelve el mismo valor para los mismos argumentos (útil para optimización).

```
DELIMITER //
```

```
CREATE FUNCTION nombre_funcion(parametro TIPO)
```

```
RETURNS TIPO_RETORNO
```

```
DETERMINISTIC
```

```
BEGIN
```

```
    DECLARE resultado TIPO_RETORNO;
```

```
    -- Lógica de cálculo
```

```
    RETURN resultado;
```

```
END;
```

```
//
```

```
DELIMITER ;
```

Funciones definidas por el usuario

Las funciones devuelven **un único valor** y pueden usarse dentro de cualquier consulta.

No pueden ejecutar INSERT/UPDATE/DELETE.

DETERMINISTIC: Siempre da el mismo resultado con la misma entrada (para almacenarla en caché). NO DETERMINISTIC cuando depende de valores RAND o fecha.

```
-- ejemplo funciones
DELIMITER //
CREATE FUNCTION salario_con_bonus(sal DECIMAL(8,2), antig INT)
RETURNS DECIMAL(8,2)
DETERMINISTIC
BEGIN
    IF antig >= 10 THEN RETURN sal * 1.10;
    ELSEIF antig >= 5 THEN RETURN sal * 1.05;
    ELSE RETURN sal;
    END IF;
END;
//
DELIMITER ;
SELECT nombre, salario, antigüedad,
        salario_con_bonus(salario, antigüedad) AS salario_final
FROM trabajadores;
```

	nombre	salario	antigüedad	salario_final
▶	Rojo Iglesias, Marta	63000.00	12	69300.00
	Gomez Corachán, Manuel	22000.00	15	24200.00
	Perez Carrillo, Iván	50400.00	5	52920.00
	Torres Marqués, Fernando	57750.00	11	63525.00
	Rubio Sánchez, María	36000.00	4	36000.00
	Llamas Rocasolano, Isabel	28000.00	13	30800.00
	Roca Benítez, Elena	35000.00	6	36750.00
	Perez Cano, Isabel	36000.00	2	36000.00
	Nualart Vives, Carlos	30000.00	10	33000.00
	Muñoz Casas, Montse	40000.00	7	42000.00

Los cursores permiten recorrer los resultados **fila por fila** de un SELECT, útil cuando la lógica no puede resolverse únicamente con SQL. MySQL permite el uso de cursores **únicamente dentro de procedimientos almacenados**.

Elementos:

- DECLARE ... CURSOR FOR: asocia el cursor a una consulta SQL.
- OPEN: inicia el cursor y lo prepara para recorrer los resultados.
- FETCH INTO: extrae la siguiente fila del cursor y asigna sus valores a variables.
- NOT FOUND handler: detecta cuándo se ha alcanzado el final del conjunto de resultados.
- CLOSE: libera los recursos asociados al cursor.

```
DECLARE nombre_cursor CURSOR FOR consulta_select;
DECLARE CONTINUE HANDLER FOR NOT FOUND SET variable_control = valor;

OPEN nombre_cursor;

cursor_loop: LOOP
    FETCH nombre_cursor INTO variable1, variable2, ...;
    IF variable_control THEN
        LEAVE cursor_loop;
    END IF;
    -- Acciones a realizar por fila
END LOOP;

CLOSE nombre_cursor;
```

Cursores

```
-- ejemplo cursor
DELIMITER //
CREATE PROCEDURE trabajadores_por_ciudad(IN p_ciudad VARCHAR(40))
BEGIN
    DECLARE v_nombre VARCHAR(50);
    DECLARE v_antig INT;
    DECLARE v_sal DECIMAL(8,2);
    DECLARE fin_cursor BOOL DEFAULT FALSE;

    -- Cursor para seleccionar trabajadores de una ciudad concreta
    DECLARE cur CURSOR FOR
        SELECT nombre, antigüedad, salario
        FROM trabajadores WHERE ciudad = p_ciudad;
    -- Handler para detectar que ya no hay más filas
    DECLARE CONTINUE HANDLER FOR NOT FOUND SET fin_cursor = TRUE;

    OPEN cur;
    bucle: LOOP
        FETCH cur INTO v_nombre, v_antig, v_sal;
        IF fin_cursor THEN LEAVE bucle;
        END IF;

        SELECT v_nombre AS nombre, v_antig AS antigüedad,
            v_sal AS salario;
    END LOOP;
    CLOSE cur;
END;
//
DELIMITER ;
```

173 • **CALL** trabajadores_por_ciudad('Madrid');

Result Grid			
Filter Rows:		Export:	
Wrap Cell Content:			
	nombre	antigüedad	salario
▶	Gomez Corachán, Manuel	15	22000.00

Excepciones

En la programación de bases de datos con MySQL, el control de excepciones permite **gestionar situaciones anómalas o imprevistas durante la ejecución** de procedimientos y bloques de código.

MySQL implementa este control mediante los **manejadores de errores** (HANDLER), que actúan ante ciertos eventos definidos, como el final de un cursor o una violación de restricción.

```
DECLARE tipo_manejador HANDLER  
FOR condición  
instrucción_o_bloque;
```

```
DELIMITER //  
CREATE PROCEDURE insertar_cliente_unico(nombre VARCHAR(50))  
BEGIN  
    DECLARE EXIT HANDLER FOR SQLEXCEPTION  
    BEGIN  
        SELECT 'Error al insertar el cliente. Verifique si ya existe.' AS mensaje_error;  
    END;  
  
    INSERT INTO clientes(nombre) VALUES (nombre);  
END;  
//  
DELIMITER ;
```

Disparadores (Triggers)

Un disparador (*trigger*) en MySQL es un objeto de base de datos que ejecuta automáticamente un bloque de instrucciones cuando ocurre un evento (INSERT, UPDATE, DELETE) sobre una tabla específica. Se utiliza para mantener la integridad, registrar auditorías o automatizar procesos derivados del cambio de datos.

Los disparadores se ejecutan de forma implícita por el sistema, no necesitan ser llamados manualmente.

Características

- Solo pueden asociarse a **una tabla concreta**.
- Se ejecutan **antes** o **después** de un evento (BEFORE o AFTER).
- El evento puede ser: INSERT, UPDATE, o DELETE.
- En MySQL, por cada combinación de BEFORE/AFTER y INSERT/UPDATE/DELETE solo puede haber **un disparador por tabla**.

Uso de las variables OLD y NEW

Estas variables permiten acceder a los valores anteriores y posteriores del registro:

- OLD: representa el valor **antes** del cambio (solo en UPDATE y DELETE).
- NEW: representa el valor **después** del cambio (solo en INSERT y UPDATE).

Sintaxis básica:

```
DELIMITER //
```

```
CREATE TRIGGER nombre_disparador
```

```
{BEFORE | AFTER} {INSERT | UPDATE | DELETE}
```

```
ON nombre_tabla
```

```
FOR EACH ROW
```

```
BEGIN
```

```
    -- Instrucciones SQL
```

```
END;
```

```
//
```

```
DELIMITER ;
```

Disparadores (Triggers)

Ejemplo: Un disparador (*trigger*) que guarda en una tabla en forma de log todas las modificaciones que se realizan en el salario de los trabajadores.

```
CREATE TABLE trabajadores_log (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    dni VARCHAR(9),  
    nombre VARCHAR(50),  
    salario_anterior DECIMAL(8,2),  
    salario_nuevo DECIMAL(8,2),  
    fecha DATETIME  
);
```

```
-- ejecución del trigger  
UPDATE trabajadores  
SET salario = salario + 2000  
WHERE dni = '11112222A';  
  
select * from trabajadores_log;
```

```
DELIMITER //  
CREATE TRIGGER trg_salario_log  
AFTER UPDATE ON trabajadores  
FOR EACH ROW  
BEGIN  
    IF OLD.salario <> NEW.salario THEN  
        INSERT INTO trabajadores_log  
        VALUES ( NULL, OLD.dni, OLD.nombre, OLD.salario, NEW.salario, NOW() );  
    END IF;  
END;  
//  
DELIMITER ;
```

	id	dni	nombre	salario_anterior	salario_nuevo	fecha
▶	1	11112222A	Rojo Iglesias, Marta	60000.00	62000.00	2025-11-20 07:02:08

¡Gracias!

A large, vibrant pink abstract graphic on the right side of the slide. It consists of a rounded rectangular shape at the top and a large, sweeping diagonal line that curves upwards and to the right, creating a sense of movement and modern design.